Reviewer Pack — Minimal Rank of Quandle Representations vs BP_ReadTwice Circ…

Entry 4f376da3f610 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 04:43:00 UTC

Minimal Rank of Quandle Representations vs BP_ReadTwice Circuit Size

Entry ID: `4f376da3f610` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-24 04:43:00 UTC

1. Conjecture

Field A (mathematical branch): Quandle Theory **Field B** (complexity object): Complexity Theory (BP_ReadTwice)

Statement:

> For a read-twice branching program P, the minimal rank of its associated quandle representation ρ(P) is O(log n) where n is the size of P, but for an IP_2 trivial BP, ρ(P) = Ω(n^2).

Rationale (proposer's reasoning):

> Quandle Theory provides a algebraic structure that can capture non-commutative properties of sets. By associating quandles with branching programs, we may expose hidden complexity in their structures, potentially leading to new insights into communication complexity.

Taxonomy category: `BP_READTWICE` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `20b9888bec8b4ab3`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the minimal rank of ρ(P) for read-twice BPs is O(log n) with at least 80% of instances having a rank within ±3 of log n, and ρ(P) for IP_2 trivial BP is ≥ Ω(n^2). The conjecture is falsified if any seed produces a rank outside these bounds.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `"Quandle Theory" AND "BP_ReadTwice circuit complexity"` - `"minimal rank quandle representation" AND "circuit size BP_ReadTwice"` - `"read-twice branching program" AND "quandle theory analysis"`

Top relevant hits considered: - [s2:2869c6b4a2f0a54083666b76ceb6d916c9ee370c] Precise and Accurate Processor Simulation - [s2:8ae61c51a45845b4307766f452a88f6958d8566c] Precise and Accurate Processor Simulation Electrical and Computer Engineering

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_bp(n):
        bp = [[random.randint(0, n-1) for _ in range(i+1)] for i in range(n)]
        return bp

    def quandle_representation(bp):
        n = len(bp)
        q = [[0] * (i+1) for i in range(n)]
        for i in range(n):
            for j in range(i+1):
                new_row = [(q[j][k] + bp[k]) % n for k in range(j+1)]
                q[j] = new_row
        return q

    def minimal_rank(q):
        rank = 0
        for row in q:
            if any(row[i] != (i * 2) % len(row) for i in range(len(row))):
                rank += 1
        return rank

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
```

```
        bp = generate_bp(n)
        q = quandle_representation(bp)
        rank = minimal_rank(q)
        results.append({
            "n": n,
            "rank": rank
        })

    mean_rank = sum(result["rank"] for result in results) / len(results)
    std_rank = math.sqrt(sum((result["rank"] - mean_rank) ** 2 for result in results) / len(result

    conjecture_holds = all(abs(rank - math.log(n)) <= 3 for n, rank in zip(n_values, [result["rank
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "minimal_rank",
        "metric_value": mean_rank,
        "instances_tested": len(results),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(seed) for seed in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_rank = sum(result["metric_value"] for result in results) / len(results)
    std_rank = math.sqrt(sum((result["metric_value"] - mean_rank) ** 2 for result in results) / le
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_rank} std={std_rank} support_fraction={support_fracti
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_fai
    else:
        print(f"RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b4aa61dc.py", line 74, in <module>
    result = run_trial(seed)
```

```
            ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b4aa61dc.py", line 46, in run_trial
    q = quandle_representation(bp)
        ^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b4aa61dc.py", line 30, in quandle_repre
    new_row = [(q[j][k] + bp[k]) % n for k in range(j+1)]
                ~~~~~~~~^~~~~~~
TypeError: unsupported operand type(s) for +: 'int' and 'list'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which prevents us from evaluating whether the conjecture is supported or falsified. | next: Debug the test code to ensure it runs successfully and produces the expected output.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	10895
2	preregistration	ollama_remote	glm4:latest	0	0	5890
3	novelty	ollama_remote	glm4:latest	0	0	4691
4	novelty	ollama_remote	glm4:latest	0	0	6294
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13154
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8573
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10748
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8735
9	judge	ollama_remote	glm4:latest	0	0	12543

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 81525 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/4f376da3f610.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/4f376da3f610.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/4f376da3f610.tar.gz` (if generated)